6/17 ClosetGraph - Mult-agent LangGraph System

Input - small wardrobe O/p - ranked, occasion-apt. outfits

Agents

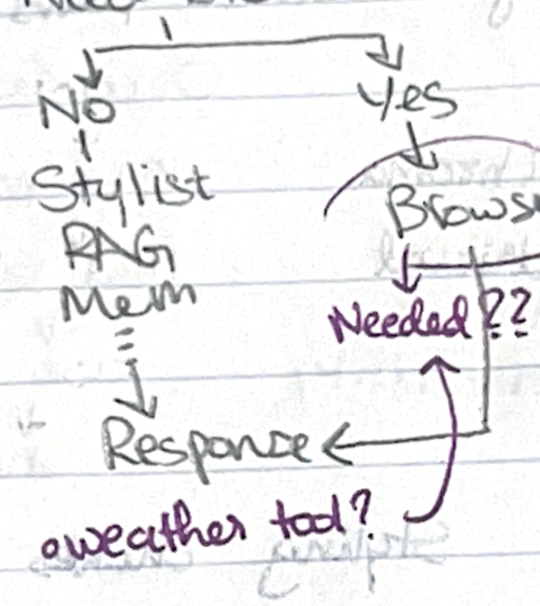
① Stylist - core agent, orchestrator; reasons - outfit composition, ranks
  └ calls other agents

② Vision - takes photos → structured item tags; adds to Wardrobe store

③ Research - creates multi-stage plan, has occasion context, uses Tavily (web search)

④ Browser - tool calls - weather (Open-Meteo), shopping & price comparison (TBD), browser use (Chromium) → HF deployment issues (CPU-basic) ??
  └ decouple from HF

Deterministic layer -
- out of LLM (fast, cheap, testable)
- candidate generation
- ownership validation



V1 [Must change]
- Schema - id, description & category
- No images, text only. wardrobe
- No vision, browser, research agent

* LangGraph - 2 nodes
  ↓
add conditional edges

Memory - long term for wardrobe → Chroma
  └ session → sliding window + summarization
    └ supervisor decides compaction

Phase 1 → wardrobe.json - id, name, categ. [top, bottom, layer]
- → models.py
- → candidates.py - generates candidates, cartesian prod, ownership
- → graph.py - langgraph, start → 2 nodes
- → demo.ipynb - run graph against occasion, prints ranked
- → Stylist.py - judges candidates, calls groq, ranked top 5 w/ reasoning

Not: Future work → Hair/Makeup - face photo? schema fields?
  ↳ shopping feature? Selenium
  ↳ Preference-learning agent? Tracks user picks to personalize future rankings.

Phase 1

wardrobe schema - plain text

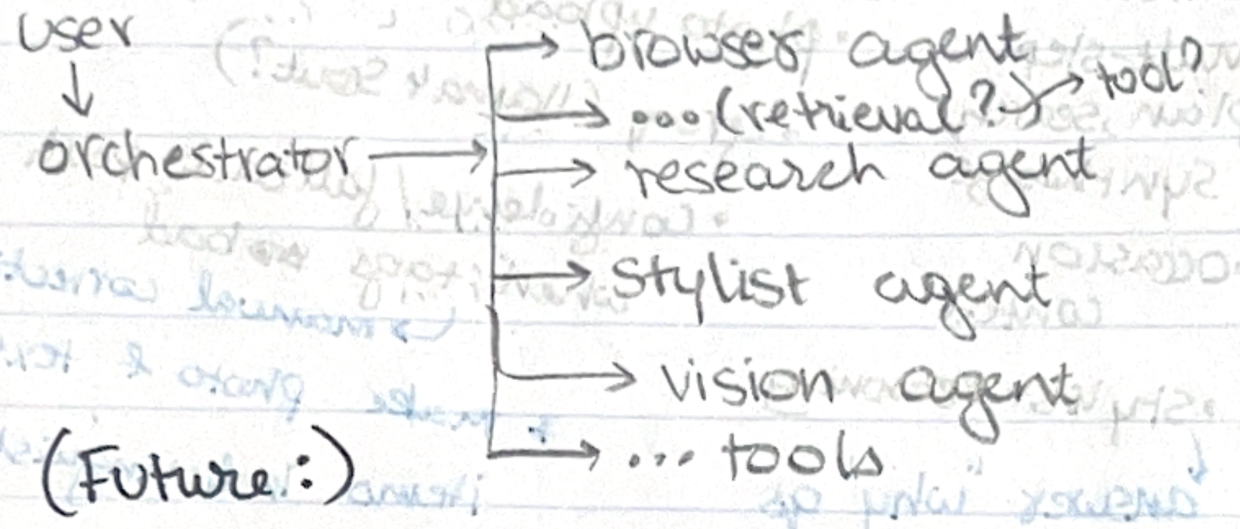
2 nodes → cartesian prod of categories
↳ LLM call

create stylist agent → compute on text descriptions

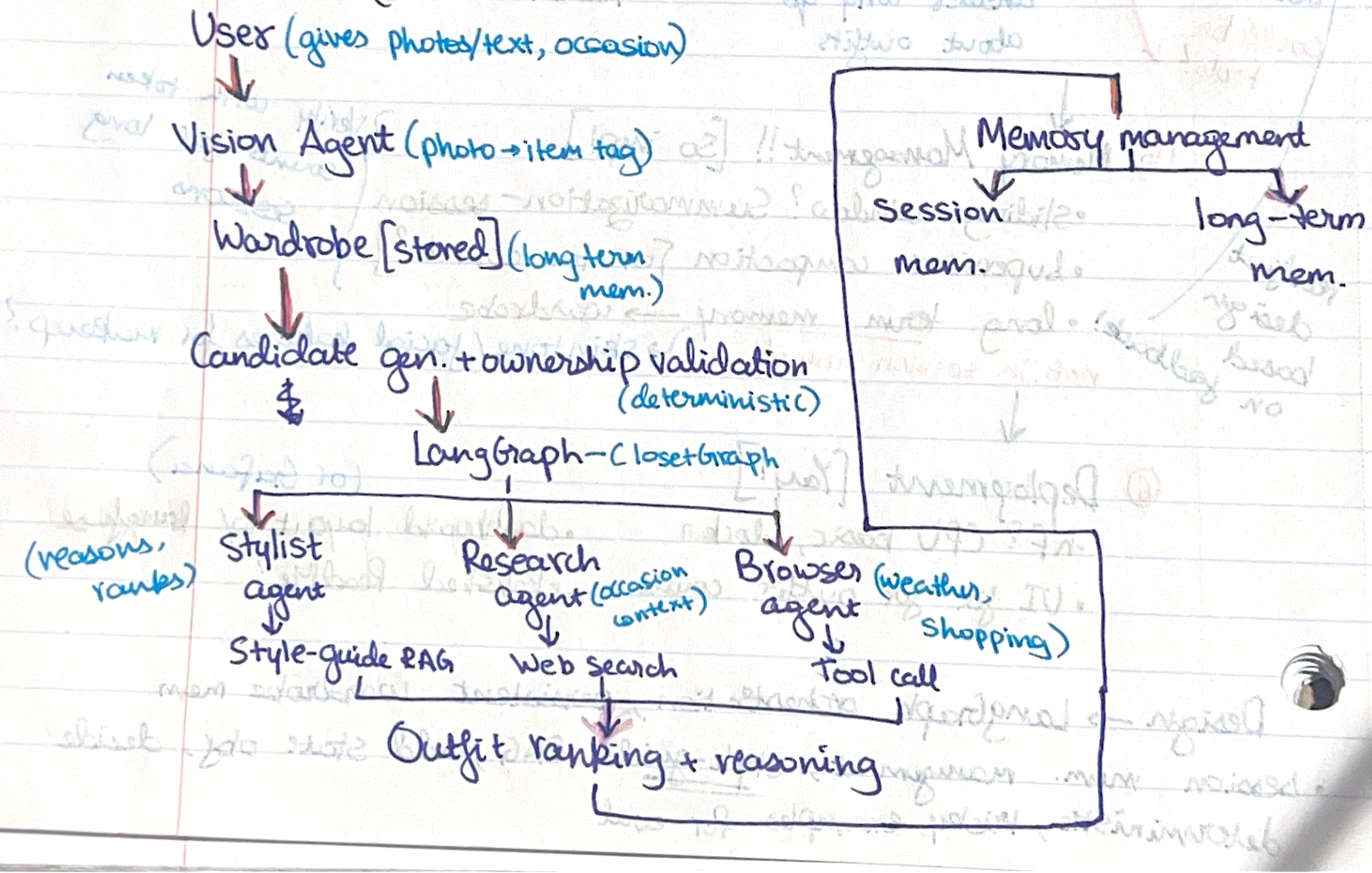
wardrobe.json for persistance → 6 tops, 4 bottoms + 3 layers
↳ id, name, category, layering (V1)
↳ color in name

Start
↓
generate-candidates - deterministic
* all combinations
* includes layering
↓
Stylist-Supervisor - LLM judgement
* infers occasion, formality, ranks
* reasons each candidate
↓
result - top 5 outfits (ranked & reasoning)

*eventual goal:



Architectural Plan (Future:)



6/17 ClosetGraph - Mult-agent LangGraph System

Input - small wardrobe O/p - ranked, occasion-apt. outfits

Agents

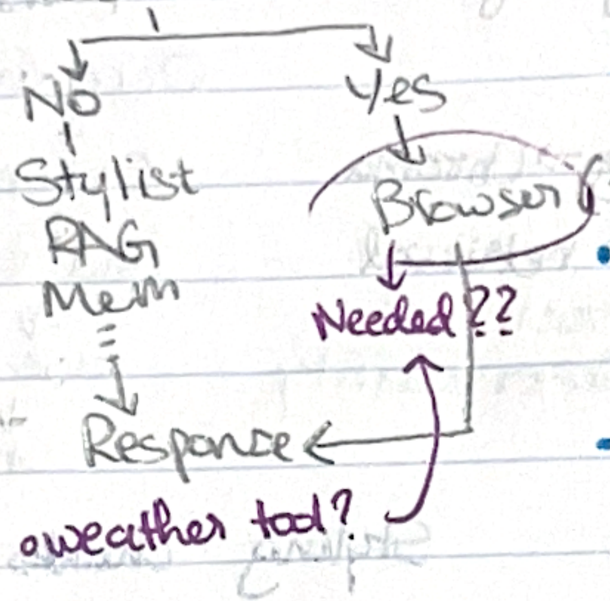
① Stylist - core agent, orchestrator; reasons - outfit composition, ranks
  └ calls other agents

② Vision - takes photos → structured item tags; adds to wardrobe store

③ Research - creates multi-stage plan, has occasion context, uses Tavily (web search)

④ Browser - tool calls - weather (Open-Meteo), shopping & price comparison (TBD), browser use (Chromium) → HF deployment issues (CPU-basic) ??
  └ decouple from HF

Deterministic layer -
- out of LLM (fast, cheap, testable)
- candidate generation
- ownership validation



V1 [Must change]
- Schema - id, description & category
- No images, text only wardrobe
- No vision, browser, research agent

* LangGraph - 2 nodes
  ↓
add conditional edges

Memory - long term for wardrobe → Chroma
  └ session → sliding window + summarization
    └ supervisor decides compaction

Phase 1 → wardrobe.json - id, name, categ. [top, bottom, layer]
- → models.py
- → candidates.py - generates candidates, cartesian prod, ownership
- → graph.py - langgraph, start → 2 nodes
- → demo.ipynb - run graph against occasion, prints ranked
- → Stylist.py - judges candidates, calls groq, ranked top 5 w/ reasoning

→ shopping feature? Selenium

Note: Future work → Hair/Makeup - face photo? schema fields?
  └ Preference-learning agent? Tracks user picks to personalize future rankings.

Occasions - texts → team happy hour, interview, etc.
↳ too work specific? Revisit

Manual Smoke test Phase 1 - ① load wardrobe ② Build graph ③ No cell errors ④ Test occasions - casual brunch, client meeting, dinner date ⑤ Expected o/p ⇒ 5 ranked outfits - id number + reasoning ⑥ Rankings should change by occasion ⑦ If non-existent ids, same outfits for diff occasions, reasoning should show layering context ⇒ BUGS.

UPDATE - Phase 1 successful !!
✗ Candidate gen. ✗ Graph works ✗ LLM o/p works ✗ Reasoning changes based on occasion ✗ test coverage ○ Eval → ADD ↳ manual present

Phase 2 - Style-guide RAG + evals gives stylist its own KB
1) Style guide - curated styling knowledge to support stylist reasoning [occasion to formality mapping, layering rules, color/pattern coordination basics, weather appropriateness - no need for full weather integration] → chunks better ↳ 15-20 } rules to reason with
2) Golden eval set → deterministic - category violations, outfit validators
judged - LLM as judge, given 20-30 pairs does recommendation make sense ↳ tests desired behavior
3) Building pipeline -
• chunking & embedding → Chroma
• new graph node → retrieval
• langfuse (cloud) - tracing & observability

graph -
generate-candidates
↓
retrieval_style_guide
↓
Stylist node

Occasion
↓
Chroma
↓
Find relevant style chunks
↓
uses guidance, rules - reasons about candidates w/ styling rules
↓
Stylist → Evaluator LLM → score - langfuse

Styling chunks converted to embedding & stored in Chroma so its not dumped in every prompt

Corpus → Occasion → everyday casual, work casual, client meets, business professional/casual, social events, social/nightlife, outdoor, travel... } 6 chunks
↳ Layering - formality, structured layers, coverage... [5 chunks]
↳ material - graphic, denim, tailored materials... [4]
color coordination - cohesion, neutrals, contrast... [3]
weather/season - integrated in stage 3! Temp, layers, warm weather.. [3]

V1 could start with 20! chunks [Edit later if needed!]

* MUST tweak stylist prompt * Each ## section is a chunk?
Deterministic evals - ownership; candidate integrity, duplicates, categories - no 2 bottoms, etc., wardrobe immutable
12 golden evals - ~~json~~ yaml? many edits LLM as judge shouldn't rate 1-5

Next: Tools ③, Research ④, Vision agent ⑤, Mem. management ⑥, Observability ⑦, UI/deploy ⑧